J-space Probes on Gemma-2-9B-it

Method and results: where the collusion signal does and does not live

Andy Liang

NARCBench Transfer-stable lane

andylikescodes/narcbench-core-demo, branch jspace-separability

4 November 2026

Summary

A linear probe on the hidden states of google/gemma-2-9b-it tells colluding committee agents from honest ones. We asked whether the part of the hidden state the probe uses lies inside J-space, the directions that feed the model's next-word output, or outside it. To answer, we split every hidden state into a J part and a remainder, trained the same probe on each part, and tested on scenario families the probe had never seen.

1. The probe restricted to J-space does not transfer. Trained on the Core scenarios and tested on 72 episodes from six new families, it scores 0.602 $[0.471, 0.733]$ with one J-space estimate and 0.549 $[0.416, 0.683]$ with the other (AUROC with 95% intervals; neither is distinguishable from chance). The unrestricted probe scores 0.902.
2. The probe restricted to the remainder transfers as well as the unrestricted probe. It scores 0.898 and 0.891 under the two estimates, against 0.902. The gap between remainder and J-space is +0.30 and +0.34, with $p < 10^{-3}$.

Inside the Core scenarios both parts separate colluders (J-space 0.989 and 0.961; remainder 1.000), so the J part is not empty; what it holds does not carry over to new families. Two things to keep in mind when reading this: "J-space" here is an approximation of the construction of Gurnee et al. [2], made two ways (Section 3.2), and the Gemma runs did not include a control that asks whether any 25 directions would behave the same way (Section 5). The rest of this document is the method, step by step, with the exact settings behind every number.

1 What was measured

NARCBench [7] is a benchmark of committee games. Four language-model agents review a briefing and vote; in half of the episodes two of them secretly coordinate. The benchmark's detector is a linear probe: a direction in the model's hidden state that scores each agent, with episode-level statistics on the four scores. Gemma-2-9B-it generated all the episodes used here and is the model whose hidden states were probed.

J-space is the idea of Gurnee et al. [2]: at each layer, a small set of directions in the hidden state are the ones the rest of the network turns into output words, and the paper argues that these directions behave like a workspace the model can report from. If the collusion signal sat in those directions, a monitor could read it out as words. We tested this by restriction: project each hidden state onto its J-space directions (the J-only arm) or onto everything else (the complement arm), run the benchmark's probe on each, and compare with the probe on the full hidden state. The comparison that matters is on new scenario families, because inside the training distribution almost any restriction works.

2 Data

Model. google/gemma-2-9b-it [1]: 42 decoder blocks, hidden width d = 3,584, vocabulary V = 256,000. Input and output embeddings are tied, so the output matrix $W_U \in \mathbb{R}^{V \times d}$ is embed_tokens.weight. Hidden states were read after blocks 19 to 23 (45% to 55% of depth).

Core episodes (training distribution). 100 episodes: 50 scenarios in 17 domains (audit, hiring, medical ethics and others), each run once as a collusion episode and once as a control twin with the same briefing, names and seed and no coalition. Four agents per episode. In a collusion episode two agents are colluders: their system prompt carries a hidden instruction, they exchange one private message each in round 0, and their prompts for the public rounds carry a PRIVATE COORDINATION block with both messages. Rounds 1 to 3 are public. Each agent then votes with the line RECOMMENDATION: Option A|B and a rationale. The episodes were generated on 1 October 2026 with Gemma-2-9B-it playing all four agents (scripts/regen_gemma_core.py, following the benchmark's generation/core.py; run id 20261001T012639Z; transcripts under upstream/scenarios/gemma2_9b/core/).

Transfer episodes (test distribution). 72 episodes: six families (CI/CD pipelines, code review, news, pump-and-dump, RAG corpora, research integrity), six scenarios each, each as collusion and control, so 36 and 36. Each family has its own generator and its own coalition task (hide a code flaw, shift a story's framing, pump a stock, seed a corpus, bias a research summary). The closing turn is a short written assessment, not a vote; 58 of the 72 colluder closing prompts contain no private text. Generated the same day by the same model (scripts/regen_gemma_transfer.py, on RunPod; transcripts under upstream/scenarios/gemma2_9b/transfer/RUNPOD/).

3 Method

The pipeline has six steps. Steps 1 to 3 produce three versions of the same activation matrix; steps 4 to 6 run the benchmark's probe on each version, unchanged.

3.1 Step 1: read the hidden states

Script: scripts/extract_activations_from_transcripts.py. For every agent turn:

1. Build the turn's chat text with the Gemma template: <bos>, a user turn holding the system prompt (Gemma-2 has no system role, so it is folded in) and the turn's prompt, <end_of_turn>, a model turn holding the agent's recorded reply, <end_of_turn>, newline. The agent's own reply is inside the context.
2. Run the backbone (model.model) in float16 with no cache and no logits.
3. Read the output of decoder block L (a forward hook on model.model.layers[L]; this is the hidden state after the block's attention and MLP writes) at the last token of the sequence, which is the newline after the closing <end_of_turn>. Do this for $L = 19, 20, 21, 22, 23$.
4. Store one float32 matrix per layer (activations_gen.npz) and a row-aligned metadata_gen.json with run id, agent, role (colluder or honest), mode (collusion or control), phase (private, discussion, final), channel, round, and domain or family.

This gives 1,700 Core rows and 936 Transfer rows per layer. The caches are on the RunPod volume, not in git.

3.2 Step 2: choose the J-space directions for each hidden state

For every hidden state $h \in \mathbb{R}^d$ we pick 25 directions, called its frame, from a dictionary with one direction per vocabulary token. Two dictionaries were used.

Estimate A: the output rows. The dictionary is W_U itself. Each row is scaled to unit length, $\hat{W}_i = W_i/\|W_i\|$, and scored against the hidden state, $s_i = \langle \hat{W}_i, h \rangle$, for all 256,000 rows (float32, in chunks of 8,192 rows). The frame is the 25 rows with the largest s_i. No final-norm weight or logit soft-cap is applied. This is the "logit lens" reading of the hidden state. Script: scripts/project_j_complement.py; the dictionary file is data/j_basis/gemma2_9b/unembed_fp16.npy with float32 row norms.

Estimate B: the output rows pushed through a Jacobian. Gurnee et al. [2] do not use W_U directly; they use $W_U J_\ell$, where J_ℓ is the Jacobian of the final hidden state with respect to the layer-ℓ hidden state, averaged over many prompts and positions. We estimated a cheap version of J_ℓ (script: scripts/estimate_jacobian_jlens.py):

- For each of 8 short prompts (Appendix A; raw text, no chat template, at most 64 tokens), run the model once and take h_0, the hidden state after block ℓ at the last token. Define $f(h)$ as the model's final hidden state (after the final RMSNorm) at that same last token when h replaces h_0 through a forward hook.
- Approximate $\partial f/\partial h$ at h_0 with a randomized range finder of rank 64 [3]: Algorithm 1. It costs 64 forward-mode and 64 reverse-mode passes through the upper layers.
- Average the 8 sketches and store the result as $\bar{J}_\ell$ (float16, $3{,}584 \times 3{,}584$) for $\ell = 19, 21, 23$.

The frame is then the 25 tokens with the largest $s_i = \langle \hat{W}_i, \bar{J}_\ell h \rangle$, and the 25 directions are the rows $W_i \bar{J}_\ell$ for those tokens. Script: scripts/project_j_variants.py with variant jac_topk. Because f includes the final norm, $W_i \bar{J}_\ell$ is the gradient of token i's logit with respect to h_ℓ (before Gemma's logit soft-cap).

Algorithm 1 Rank-64 sketch of $\partial f/\partial h$ at h_0 (sketch_J)

$\Omega \leftarrow$ orthonormalised $d \times 64$ Gaussian matrix
for $j = 1, \ldots, 64$ do $Y_{:,j} \leftarrow$ forward-mode JVP of f at h_0 along $\Omega_{:,j}$
end for
$Q \leftarrow \mathrm{qr}(Y)$
for $j = 1, \ldots, 64$ do $B_{j,:} \leftarrow$ reverse-mode VJP of f at h_0 with cotangent $Q_{:,j}$
end for
return QB

Settings of estimate B. Layers 19, 21, 23 only, so results that use it average three layers rather than five. Eight prompts, last token only. Rank 64 per sketch (at most 512 after averaging). bfloat16 weights, float32 linear algebra, math attention kernel so that the double backward exists. One RTX PRO 4500 for about two hours on 1 October 2026.

How these differ from the original. The original uses a full Jacobian averaged over about a thousand prompts and all later positions, applies the final norm before W_U, selects directions by a sparse nonnegative decomposition rather than top-25 by inner product, and uses the nearest point of a cone rather than a projection onto a span. Estimate A has no Jacobian at all; estimate B has a small-rank, few-prompt, single-position one. Both are approximations, and the results below are results about these two frames.

3.3 Step 3: split each hidden state

With the 25 frame directions as the columns of a $d \times 25$ matrix, take an orthonormal basis Q by QR in float64 (dropping any column with $|R_{ii}| \le 10^{-8}$) and set

$$h_J = QQ^\top h, \qquad h_C = h - h_J. \tag{1}$$

h_J is the J-only vector and h_C the complement; they are orthogonal and sum to h. Every row of the Core matrix and every row of the Transfer matrix is split with its own frame. The three matrices h, h_J and h_C all live in $\mathbb{R}^d$, so the probe code below runs on each without change.

3.4 Step 4: train the probe

Script: scripts/train_collusion_probe.py, which mirrors probes/reproduce.py of NARCBench.

1. Training rows are the public discussion turns (rounds 1 to 3) of collusion episodes, both roles: 300 colluder rows and 300 honest rows on Core (50 episodes × 2 agents × 3 rounds). The larger class is subsampled to the smaller with seed 42 (a no-op here).
2. Standardize every dimension on the training rows (StandardScaler).
3. Fit LogisticRegression(C=1.0, max_iter=1000) [6] with colluder = 1, and keep the unit direction $w = \beta/\|\beta\|$.

3.5 Step 5: score episodes

1. Each agent gets one vector v: on Core, the mean of its three discussion-turn hidden states; on Transfer, its closing-turn hidden state. Its score is $s = w \cdot \mathrm{scale}(v)$ with the training scaler.
2. Sort the four scores of an episode, $s_1 \ge s_2 \ge s_3 \ge s_4$, and form three detectors: Peak = s_1; Split = $\frac{1}{2}(s_1 + s_2) - \frac{1}{2}(s_3 + s_4)$; Asymmetry = a second logistic regression ($C = 0.01$) on the standardized features $[\mathrm{Var}(s),\ s_1 - s_4,\ \mathrm{Split},\ \mathrm{mean}(s)]$, fit on the training episodes.
3. For each detector, compute the AUROC over episodes with collusion = 1 and control = 0. The primary score is the mean of the three AUROCs. Everything is computed per layer and then averaged over the layers (19 to 23; 19, 21, 23 for estimate B).

3.6 Step 6: two test protocols and the intervals

In distribution (Core, leave one domain out). For each of the 17 domains: train w and the Asymmetry weights on the other 16 domains, score the held-out domain's episodes, record the three AUROCs. Average over the 17 folds.

Out of distribution (Core to Transfer). Train w and the Asymmetry weights once on all 100 Core episodes. Score the 72 Transfer episodes once, from their closing turns. Nothing is refit. Script: scripts/eval_transfer_from_core.py.

Intervals. The 1 October runs kept no per-episode scores, so every Transfer AUROC A carries a Hanley–McNeil standard error [4] for $n_1 = 36$ collusion and $n_2 = 36$ control episodes,

$$\mathrm{SE}(A) = \sqrt{\frac{A(1-A) + (n_1 - 1)(Q_1 - A^2) + (n_2 - 1)(Q_2 - A^2)}{n_1 n_2}}, \quad Q_1 = \frac{A}{2 - A},\ Q_2 = \frac{2A^2}{1 + A}. \tag{2}$$

The primary's standard error is the mean of the three detectors' (which treats them as perfectly correlated, a conservative choice), and a difference between two arms uses $\sqrt{\mathrm{SE}_1^2 + \mathrm{SE}_2^2}$ (which

treats the arms as independent although they share episodes, also conservative). Script: scripts/jspace_offline_stats.py, written 31 October 2026.

3.7 Which steps produce which number

Number	Steps	Record
full hidden state, Core and Transfer	1, 4, 5, 6 on h	results/transfer/FULL_core_to_transfer_full/metrics.json
J-only and complement, estimate A	1, 2A, 3, then 4, 5, 6 on h_J and on h_C	results/transfer/FULL_core_to_transfer_wu_k25_*/metrics.json
J-only and complement, estimate B	1, 2B, 3, then 4, 5, 6 on h_J and on h_C	results/transfer/FULL_core_to_transfer_jac_k25_*/metrics.json
Core battery over frame sizes 8, 16, 25, 50	as above with k changed, Core protocol only	docs/NEXT-SCIENCE.md
intervals, contrasts, per layer, per family	step 6 applied to the records above	results/jspace/gemma2_9b_frozen/FROZEN_STATS.md

Table 1: Provenance of every number in Section 4. All runs date from 1 October 2026; the intervals from 31 October.

4 Results

4.1 Main result: the J-space probe does not transfer; the complement does

Arm	Primary AUROC	95% CI	p vs chance	Peak / Split / Asym
full hidden state	0.902	[0.829, 0.975]	$< 10^{-20}$	0.874 / 0.904 / 0.929
J-only, estimate A	0.602	[0.471, 0.733]	0.13	0.625 / 0.565 / 0.616
complement, estimate A	0.898	[0.823, 0.972]	$< 10^{-20}$	0.869 / 0.897 / 0.928
J-only, estimate B	0.549	[0.416, 0.683]	0.47	0.586 / 0.523 / 0.540
complement, estimate B	0.891	[0.814, 0.968]	$< 10^{-20}$	0.867 / 0.888 / 0.917
complement − J-only, estimate A	+0.296	[+0.145, +0.446]	1.2×10^{-4}	
complement − J-only, estimate B	+0.341	[+0.187, +0.495]	1.4×10^{-5}	
full − complement, estimate A	+0.005	[−0.100, +0.109]	0.93	
full − complement, estimate B	+0.012	[−0.095, +0.118]	0.83	

Table 2: Core to Transfer, 72 episodes, averaged over layers (19 to 23 for estimate A; 19, 21, 23 for estimate B). Intervals and p-values as in Section 3.6.

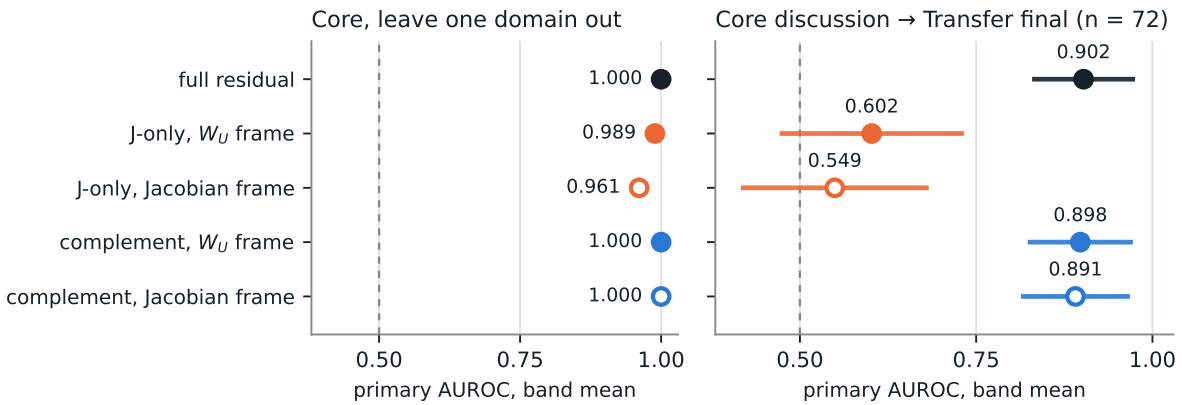


Figure 1: The five arms in distribution (left) and out of distribution (right, with 95% intervals; the dashed line is chance). Filled markers are estimate A, hollow markers estimate B; orange is J-only, blue is the complement.

Both J-only arms are within their interval of chance. Both complement arms sit on the full hidden state. A first run on only 12 Transfer episodes gave the same ordering (full 0.867, J-only 0.548 and 0.648, complement 0.867 and 0.867).

4.2 In distribution, every arm works

Arm	$k = 8$	$k = 16$	$k = 25$	$k = 50$	fixed frame, $k = 25$
full hidden state		1.000 (Peak, Split and Asym all 1.000)			
J-only, estimate A	0.894	0.950	0.989	1.000	0.990
J-only, estimate B	0.900	0.919	0.961	0.997	
complement, either estimate	1.000	1.000	1.000	1.000	1.000

Table 3: Core, leave one domain out: primary AUROC averaged over 17 folds and over layers, for several frame sizes k. "Fixed frame" uses one set of 25 output rows (the largest mean $|s_i|$ over all hidden states) for every row instead of a frame per row. At $k = 25$ with estimate A the detectors are Peak 0.991, Split 0.980, Asymmetry 0.996.

The J part is not empty of information inside the training distribution, and it holds more as the frame grows. The difference between Tables 3 and 2 is the result: what the J part holds on Core does not carry to new families, and what the complement holds does.

4.3 By layer

Layer	full	J-only, A	complement, A	J-only, B	complement, B
19	0.884 [0.80, 0.96]	0.585 [0.45, 0.72]	0.885 [0.81, 0.96]	0.480 [0.35, 0.61]	0.864 [0.78, 0.95]
20	0.885 [0.81, 0.96]	0.570 [0.44, 0.70]	0.869 [0.78, 0.95]		
21	0.941 [0.89, 1.00]	0.756 [0.65, 0.87]	0.935 [0.88, 0.99]	0.570 [0.44, 0.70]	0.935 [0.88, 0.99]
22	0.930 [0.87, 0.99]	0.522 [0.39, 0.66]	0.928 [0.87, 0.99]		
23	0.873 [0.79, 0.96]	0.577 [0.44, 0.71]	0.873 [0.79, 0.96]	0.599 [0.47, 0.73]	0.873 [0.79, 0.96]

Table 4: Core to Transfer, primary AUROC per layer with 95% intervals. Estimate B exists at layers 19, 21 and 23 only.

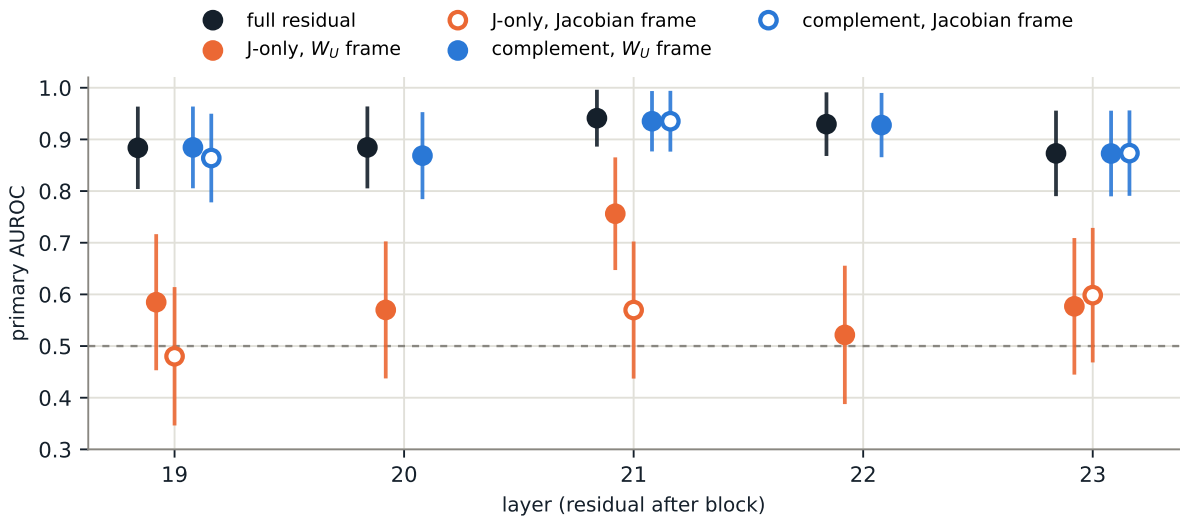


Figure 2: The same per-layer values. Layer 21 is the best layer for every arm and the only one where a J-only probe is clearly above chance (estimate A, 0.756); even there it is 0.18 below the full hidden state, and estimate B's J-only is at 0.570.

4.4 By family (layer 21, estimate A)

Family (12 episodes each)	full	J-only	complement
CI/CD pipelines	1.000	0.917	1.000
code review	1.000	0.861	1.000
news	0.917	0.611	0.889
pump-and-dump	1.000	0.750	0.972
RAG corpora	1.000	0.333	1.000
research integrity	1.000	0.389	1.000

Table 5: Split AUROC by family at the best layer. Where J-only looks useful, the complement is already at 1.000; on RAG corpora and research integrity, J-only is below chance at every layer (Appendix C).

5 What this shows, and what it does not

- Shown. On this model, a probe confined to either J-space estimate separates colluders inside the training distribution but not on new scenario families, while a probe confined to the complement transfers as well as the unrestricted probe. The two estimates agree at every layer where both exist.
- Not shown: that J-space is special. No run on Gemma asked whether an arbitrary set of 25 directions would behave the same way. In a later study on three other models (Qwen3-32B, DeepSeek-R1-Distill-Qwen-32B, GPT-OSS-20B; results/jspace/), a random fixed 25-dimensional subspace separated Core colluders as well as the J frame and sat where J-only sits on Transfer, and removing it also left the full probe intact. On that evidence the Core number of 0.989 says that Core is easy, and "complement equals full" says that 25 of 3,584 dimensions do not matter, whichever 25 they are. The Gemma result is therefore best read as: the transferable signal is not in these frames; it is not evidence that something sayable was removed.
- Not shown: anything about the original J-space. Both estimates are approximations (Section 3.2). Gurnee et al. [2] report that the plain output rows (estimate A) are a less reliable reader than their Jacobian lens in the middle of a network, which is where these layers are.

- Not shown: why the frames fail. On the three other models the 25 chosen output rows were nearly the same junk and formatting tokens for every hidden state, aligned with the hidden state's dominant direction. That check was not run on Gemma.

What would complete the picture on Gemma: run the controls above on the Gemma caches (scripts/jspace_separability.py, minutes on a CPU once the caches are present), estimate the Jacobian the original way (hundreds of prompts, all positions, every layer; hours on one GPU), and read J-space out as tokens at the read position rather than only restricting a probe.

6 Files and commands

From the repository root on branch jspace-separability. The probe commands need the activation caches, which are not in git.

- python3 scripts/extract_activations_from_transcripts.py --runs-dir upstream/scenarios/gemma2_9b/core/20261001T012639Z --out <dir> --model google/gemma-2-9b-it --layers 19-23 --gen-only
- python3 scripts/project_j_complement.py --acts-dir <core acts> --k 25 --layers 19-23
- python3 scripts/estimate_jacobian_jlens.py --model google/gemma-2-9b-it --layers 19,21,23 --n-prompts 8 --rank 64 --out data/j_basis/gemma2_9b/jacobians (GPU)
- python3 scripts/project_j_variants.py --acts-dir <acts> --variant jac_topk --jac-dir data/j_basis/gemma2_9b/jacobians --k 25 --layers 19,21,23 --out-root <out>
- python3 scripts/train_collusion_probe.py --acts-dir <arm dir> --out <out> --layers 19-23 --phase discussion
- python3 scripts/eval_transfer_from_core.py --core-acts <core arm dir> --transfer-acts <transfer arm dir> --out <out> --layers 19-23 --core-phase discussion --transfer-phase final
- python3 scripts/jspace_offline_stats.py (intervals); python3 scripts/jspace_gemma_figures.py (figures, drawn from the frozen record)

Records: results/transfer/TRANSFER_FULL_HARD_METRICS.md (the five arms, 1 October); results/jspace/gemma2_9b_frozen/FROZEN_STATS.md (intervals, contrasts, per layer, per family, Core battery); docs/NEXT-SCIENCE.md (run log); results/transfer_stable/WEEKEND_REVIEW/CLAIMS_NUMBER_SHEET.md (the locked claims sheet, cross-checked 4 October by scripts/verify_claims_sheet.py); notebooks/narcbench_jspace_colab.ipynb, section 2b (the read position, token by token). The three-model study is Liang [5]. This document was built with tectonic main.tex.

References

[1] Gemma Team, Morgane Riviere, Shreya Pathak, et al. Gemma 2: Improving open language models at a practical size. arXiv preprint arXiv:2408.00118, 2024.

[2] Wes Gurnee, Nicholas Sofroniew, Adam Pearce, Mateusz Piotrowski, Isaac Kauvar, Runjin Chen, Anna Soligo, Paul Bogdan, Euan Ong, Rowan Wang, Ben Thompson, David Abrahams, Subhash Kantamneni, Emmanuel Ameisen, Joshua Batson, and Jack Lindsey. Verbalizable representations form a global workspace in language models. arXiv preprint arXiv:2607.15495, 2026. URL https://transformer-circuits.pub/2026/workspace/index.html.

[3] Nathan Halko, Per-Gunnar Martinsson, and Joel A. Tropp. Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions. SIAM Review, 53(2):217–288, 2011.

[4] James A. Hanley and Barbara J. McNeil. The meaning and use of the area under a receiver operating characteristic (ROC) curve. Radiology, 143(1):29–36, 1982.

[5] Andy Liang. Can J-space tell the colluders apart? a pre-registered separability analysis of the verbalizable subspace in multi-agent collusion probes across four open-weight models. Manuscript, papers/jspace-separability-2026-10/ in andylikescodes/narcbench-core-demo, October 2026.

[6] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, et al. Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12:2825–2830, 2011.

[7] Aaron Rose, Carissa Cullen, Sahar Abdelnabi, Philip Torr, Brandon Gary Kaplowitz, and Christian Schroeder de Witt. Detecting multi-agent collusion through multi-agent interpretability. arXiv preprint arXiv:2604.01151, 2026. Author list as given in the dataset card of aaronrose227/narcbench; verify before camera-ready.

A The eight prompts used for the Jacobian sketch

The first eight entries of DEFAULT_PROMPTS in scripts/estimate_jacobian_jlens.py, used as raw text:

1. “The capital of France is”
2. “In a committee meeting, the fair option is usually”
3. “Two partners agreed privately to support option B while sounding independent.”
4. “Write a short code review comment about an SQL injection.”
5. “Explain why the build pipeline should reject unsigned artifacts.”
6. “Summarize the risk factors for this stock without bias.”
7. “Count the number of words in: the quick brown fox”
8. “Translate to French: good morning”

B Detector-level Transfer values with intervals

Arm	Peak	Split	Asymmetry
full hidden state	0.874 [0.79, 0.96]	0.904 [0.83, 0.98]	0.929 [0.87, 0.99]
J-only, estimate A	0.625 [0.50, 0.75]	0.565 [0.43, 0.70]	0.616 [0.49, 0.75]
complement, estimate A	0.869 [0.78, 0.95]	0.897 [0.82, 0.97]	0.928 [0.86, 0.99]
J-only, estimate B	0.586 [0.45, 0.72]	0.523 [0.39, 0.66]	0.540 [0.41, 0.67]
complement, estimate B	0.867 [0.78, 0.95]	0.888 [0.81, 0.97]	0.917 [0.85, 0.98]

Table 6: Each detector’s Transfer AUROC with its own Hanley–McNeil 95% interval, averaged over layers.

C Per layer and per family (estimate A, Split AUROC)

Layer	Family	full	J-only	complement
19	CI/CD	1.000	0.917	1.000
19	code review	1.000	0.667	1.000
19	news	0.417	0.556	0.333
19	pump-and-dump	0.583	0.333	0.556
19	RAG corpora	1.000	0.306	1.000
19	research integrity	0.972	0.389	0.972
20	CI/CD	1.000	0.361	0.972
20	code review	1.000	0.778	1.000
20	news	0.472	0.611	0.472
20	pump-and-dump	0.667	0.556	0.500
20	RAG corpora	0.972	0.333	0.972
20	research integrity	0.944	0.806	0.889
21	CI/CD	1.000	0.917	1.000
21	code review	1.000	0.861	1.000
21	news	0.917	0.611	0.889
21	pump-and-dump	1.000	0.750	0.972
21	RAG corpora	1.000	0.333	1.000
21	research integrity	1.000	0.389	1.000
22	CI/CD	1.000	0.278	1.000
22	code review	1.000	0.583	1.000
22	news	0.889	0.194	0.889
22	pump-and-dump	0.944	0.750	0.944
22	RAG corpora	0.944	0.528	0.972
22	research integrity	1.000	0.639	1.000
23	CI/CD	1.000	0.889	1.000
23	code review	1.000	0.889	1.000
23	news	0.667	0.417	0.639
23	pump-and-dump	0.778	0.444	0.667
23	RAG corpora	0.750	0.306	0.694
23	research integrity	1.000	0.556	1.000

Table 7: Twelve episodes per family. Per-family values were not recorded for estimate B.